

مقترح مشروع تدريبي — نسخة معمارية محسنة

# Job Board API

منصة توظيف Backend مبنية بمعمارية Clean Architecture و CQRS،  
مصممة لتكون أقوى تقنيًا من نماذج مشاريع التخرج التقليدية من خلال Rich  
Domain Model واختبارات حقيقية وأمان مطبق افتراضيًا

CQRS + MediatR

Entity Framework Core

C# 12

ASP.NET Core Web API

JWT Authentication

SQL Server

AutoMapper

FluentValidation

xUnit

## المحتويات

---

|    |                                     |
|----|-------------------------------------|
| ٢  | ١. نظرة عامة على المشروع            |
| ٣  | ٢. لماذا هذه المعمارية بالذات       |
| ٤  | ٣. البنية المعمارية (Architecture)  |
| ٥  | ٤. متطلبات المشروع (Requirements)   |
| ٦  | ٥. الميزات الأساسية (Core Features) |
| ٨  | ٦. ميزة الـ Matching Score          |
| ٩  | ٧. الأمان والصلاحيات                |
| ٩  | ٨. استراتيجية الاختبار              |
| ١٠ | ٩. خطة التنفيذ المقترحة             |

## ١. نظرة عامة على المشروع

Job Board API هو مشروع Backend تدريبي متكامل، الهدف منه بناء منصة توظيف تربط بين الشركات الباحثة عن موظفين والمرشحين الباحثين عن فرص عمل. المشروع مصمم عمدًا ليكون بعمق تقني حقيقي بدل الاتساع الأفقي في عدد الميزات — يعني بدل ما نغطي عشرين موديول بشكل سطحي، نركز على عدد أقل من الموديولات لكن كل واحد فيهم مبني بجودة إنتاجية (production-grade).

### الفكرة الأساسية

المشروع ده مش مجرد CRUD API. الهدف إنه يعكس فهم حقيقي لـ Clean Architecture وCQRS ومبادئ Domain-Driven Design، بشكل يميزه بوضوح عن مشاريع التخرج النمطية اللي بتركز على الكمية بدل الجودة.

## ٢. لماذا هذه المعمارية بالذات

بعد تحليل عدة مشاريع تخرج مشابهة (HR Systems, Examination Systems، إلخ)، لاحظنا نمط متكرر: معمارية طبقات كويسة على الورق، لكن التنفيذ الفعلي فيه نفس المشاكل دائماً. الجدول التالي يلخص الفرق بين النمط الشائع والنهج المتبع في هذا المشروع.

| المعيار                 | النمط الشائع في مشاريع التخرج                       | Job Board API                                       |
|-------------------------|-----------------------------------------------------|-----------------------------------------------------|
| الـ Domain Model        | <b>Anemic — مجرد Properties بدون منطق</b>           | <b>Rich — Entities</b> تحتوي على قواعد العمل بنفسها |
| طبقة الـ Application    | <b>Service Layer عادي</b>                           | <b>CQRS منفصل (Commands / Queries) عبر MediatR</b>  |
| الحماية (Authorization) | <b>جزئية — بعض الـ Controllers بدون [Authorize]</b> | افتراضية على مستوى المشروع كله                      |
| الاختبارات              | <b>غير موجودة غالباً</b>                            | <b>Unit Tests لمنطق الـ Domain الأساسي</b>          |
| نطاق الميزات            | <b>واسع جداً مقابل وقت التنفيذ المتاح</b>           | محدود ومرکز، بعمق تقني أعلى                         |

### ٣. البنية المعمارية (Architecture)

يتبع المشروع Clean Architecture بخمس طبقات، حيث تتجه كل الاعتماديات نحو الداخل باتجاه طبقة الـ Domain:

```
JobBoard/
├── src/
│   ├── Domain/                → Entities + غنية بالسلوك Value Objects
│   │   ├── Entities/
│   │   ├── ValueObjects/
│   │   └── Interfaces/
│   ├── Application/          → CQRS: Commands + Queries + Handlers
│   │   ├── Features/
│   │   │   ├── JobPostings/
│   │   │   │   ├── Commands/
│   │   │   │   └── Queries/
│   │   │   └── Candidates/
│   │   ├── Validators/       → FluentValidation لكل Command
│   │   └── Mappings/          → AutoMapper Profiles
│   ├── Infrastructure/        → EF Core, Repositories, External Services
│   ├── API/                   → Controllers + Middleware + [Authorize]
│   └── Tests/                  → Unit + Integration Tests
```

#### مبدأ اتجاه الاعتماديات

الـ Domain لا يعتمد على أي طبقة أخرى إطلاقاً. الـ Application يعتمد على الـ Domain فقط. الـ Infrastructure ينفذ الـ Interfaces المعرّفة في الـ Domain (Dependency Inversion). الـ API يجمع كل شيء عبر Dependency Injection. هذا الترتيب يضمن أن منطق العمل الأساسي يبقى قابلاً للاختبار ومستقلاً عن قاعدة البيانات أو الإطار المستخدم.

## ٤. متطلبات المشروع (Requirements)

### المتطلبات الوظيفية (Functional Requirements)

| الرمز | المتطلب                                                                             | الأولوية |
|-------|-------------------------------------------------------------------------------------|----------|
| FR-01 | تسجيل ودخول المستخدمين (شركة / مرشح) عبر JWT                                        | أساسي    |
| FR-02 | إنشاء الشركة لإعلانات وظيفية (Job Postings) وتعديلها وحذفها                         | أساسي    |
| FR-03 | تقديم المرشح على وظيفة عبر Job Application                                          | أساسي    |
| FR-04 | حساب Matching Score تلقائي بين ملف المرشح والوظيفة                                  | أساسي    |
| FR-05 | تتبع حالة الطلب عبر مراحل واضحة (Applied → Reviewed → Interview → Offer → Rejected) | أساسي    |
| FR-06 | بحث وفلتر الوظائف (حسب المهارات، الموقع، نوع الدوام)                                | مهم      |
| FR-07 | لوحة تحكم للشركة تعرض عدد المتقدمين وترتيبهم حسب Matching Score                     | مهم      |
| FR-08 | إشعارات عند تغير حالة الطلب                                                         | إضافي    |
| FR-09 | رفع السيرة الذاتية (CV) كمرفق بالطلب                                                | مهم      |

### المتطلبات غير الوظيفية (Non-Functional Requirements)

| الرمز  | المتطلب                                                                               |
|--------|---------------------------------------------------------------------------------------|
| NFR-01 | كل Endpoint محمي بـ Authorization افتراضياً، إلا ما هو معلّم صراحة كـ Public          |
| NFR-02 | كل Command له Validator مستقل باستخدام FluentValidation                               |
| NFR-03 | تغطية اختبارات (Unit Tests) لمنطق الـ Domain الحرج، وعلى رأسه Matching Score          |
| NFR-04 | معالجة أخطاء موحدة عبر Global Exception Middleware                                    |
| NFR-05 | توثيق الـ API عبر Swagger / OpenAPI                                                   |
| NFR-06 | استجابة الاستعلامات الأساسية في أقل من ٣٠٠ ميلي ثانية على بيانات تجريبية معقولة الحجم |

## ٥. الميزات الأساسية (Core Features)

### إدارة الحسابات والمصادقة

تسجيل دخول وتسجيل حسابات جديدة بنوعين من المستخدمين: صاحب عمل (Employer) ومرشح (Candidate)، مع إصدار JWT يحمل الدور (Role) ضمن الـ Claims. كل طلب لاحق يُتحقق منه عبر Middleware المصادقة قبل الوصول لأي Handler.

### إدارة إعلانات الوظائف (Job Postings)

الشركة تقدر تنشئ إعلان وظيفي يحدد فيه العنوان، الوصف، المهارات المطلوبة، نطاق الراتب، ونوع الدوام. كل عملية إنشاء أو تعديل تمر عبر Command Handler مستقل، مع Validator يمنع بيانات ناقصة أو غير منطقية (زي نطاق راتب بحد أدنى أكبر من الحد الأقصى).

### تقديم الطلبات وتتبعها (Job Applications)

المرشح يقدر يقدم على وظيفة مرة واحدة فقط لكل إعلان (قاعدة عمل محمية داخل الـ Domain نفسه، مش في الـ Service). الطلب يمر بحالات محددة سلفاً، والانتقال بين الحالات يتم عبر Method داخل الـ Entity مش عبر تعديل مباشر لحقل الحالة.

### لوحة تحكم الشركة

عرض كل المتقدمين على وظيفة معينة مرتبين حسب Matching Score تنازلياً، مع إمكانية الفلترة حسب حالة الطلب. الاستعلام هنا Read-only بالكامل ويُنفذ عبر Query منفصل عن مسار الكتابة (فصل واضح بين القراءة والكتابة).

### البحث والفلتر

بحث نصي في عنوان الوظيفة والوصف، مع فلاتر إضافية للموقع ونوع الدوام والمهارات. مبني كـ Query واحد قابل للتوسعة بمعاملات اختيارية بدل عدة Endpoints منفصلة لكل نوع فلتر.

## ٦. ميزة ال Matching Score

هذه الميزة هي القلب التقني للمشروع، وهي التي يتفرّقه عن أي CRUD API عادي. الفكرة: حساب درجة توافق (Score من ٠ إلى ١٠٠) بين ملف المرشح وإعلان الوظيفة، بناءً على قواعد واضحة قابلة للتفسير (Rule-Based) كخطوة أولى قبل التوسع لاحقاً نحو نهج مبني على الذكاء الاصطناعي.

### عوامل الحساب المقترحة

| العامل                       | الوزن | آلية الحساب                                |
|------------------------------|-------|--------------------------------------------|
| تطابق المهارات               | ٥٠%   | نسبة المهارات المشتركة بين المرشح والوظيفة |
| سنوات الخبرة                 | ٢٥%   | مقارنة خبرة المرشح بالحد الأدنى المطلوب    |
| التوافق الجغرافي ونوع الدوام | ١٥%   | تطابق الموقع أو قبول العمل عن بُعد         |
| مستوى التعليم                | ١٠%   | مقارنة المؤهل مقابل المتطلب الأدنى إن وُجد |

### لماذا Rule-Based أولاً

البدء بمنطق قواعد واضح (بدون Machine Learning) يبيخلي النتيجة قابلة للتفسير والاختبار بسهولة (Unit Testing مباشر)، وهو تحديداً المدخل المخطط له كخطوة تمهيدية نحو التوسع لاحقاً في اتجاه ال AI/ML.

مثال مبسّط على المنطق داخل ال Domain:

```
public sealed class MatchingScore
{
    public decimal Value { get; }

    private MatchingScore(decimal value) => Value = value;

    public static MatchingScore Calculate(
        CandidateProfile candidate, JobPosting job)
    {
        var skillsScore = CalculateSkillsOverlap(candidate, job) * 0.50m;
        var experience = CalculateExperienceFit(candidate, job) * 0.25m;
        var location = CalculateLocationFit(candidate, job) * 0.15m;
        var education = CalculateEducationFit(candidate, job) * 0.10m;

        var total = skillsScore + experience + location + education;
        return new MatchingScore(Math.Round(total, 2));
    }
}
```



## ٧. الأمان والصلاحيات

واحدة من أوضح المشاكل التي تظهر في مشاريع مشابهة هي نسيان حماية بعض الـ Endpoints. في هذا المشروع، الحماية مطبقة بشكل افتراضي على مستوى الإعداد العام للمشروع، مش على كل Controller لوحده.

```
// Program.cs
builder.Services.AddControllers(options =>
{
    var policy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
    options.Filters.Add(new AuthorizeFilter(policy));
});

// Any public endpoint must opt out explicitly:
[AllowAnonymous]
[HttpPost("login")]
```

### الفرق العملي

بهذا الأسلوب، أي Endpoint جديد يُضاف بالخطأ بدون تفكير بيبقى محمي تلقائياً، بدل ما يبقى مكشوف لحد ما حد يتذكر يحطله [Authorize].

## الأدوار (Roles)

| الدور     | الصلاحيات                                                          |
|-----------|--------------------------------------------------------------------|
| Employer  | إنشاء/تعديل/حذف إعلانات الوظائف، عرض المتقدمين، تحديث حالة الطلبات |
| Candidate | عرض الوظائف، تقديم الطلبات، متابعة حالة طلباته الخاصة فقط          |

## ٨. استراتيجية الاختبار

الاختبارات مش خطوة إضافية بعد الانتهاء، بل جزء من كل Feature من أول يوم. التركيز الأساسي على اختبار منطق الـ Domain لأنه الجزء الأكثر أهمية والأسهل اختباراً لأنه معزول عن قاعدة البيانات والـ HTTP.

| النوع             | ما الذي يُختبر                                   | مثال                                                               |
|-------------------|--------------------------------------------------|--------------------------------------------------------------------|
| Unit Tests        | منطق الـ Domain المعزول                          | حساب Matching Score بمدخلات مختلفة، رفض تقديم مكرر على نفس الوظيفة |
| Unit Tests        | الـ Validators                                   | رفض Command بحقل راتب سالب أو تاريخ غير منطقي                      |
| Integration Tests | Command/Query Handlers مع قاعدة بيانات In-Memory | التأكد من حفظ الطلب فعلياً وربطه بالمرشح الصحيح                    |

## ٩. خطة التنفيذ المقترحة

| المرحلة | المحتوى                                                                 |
|---------|-------------------------------------------------------------------------|
| ١       | إعداد المشروع بالطبقات الخمس + Domain Entities الأساسية (بدون منطق بعد) |
| ٢       | بناء Authentication و [Authorize] + JWT الافتراضي على مستوى المشروع     |
| ٣       | Job Postings: Commands/Queries + Validators + Tests                     |
| ٤       | Job Applications: منطق الحالات داخل الـ Entity + منع التكرار            |
| ٥       | تنفيذ Matching Score ≤ Domain Service مستقل + Unit Tests شاملة له       |
| ٦       | لوحة تحكم الشركة + البحث والفلتر                                        |
| ٧       | مراجعة شاملة: تغطية الاختبارات، توثيق Swagger، تنظيف الكود              |

### ملاحظة ختامية

الهدف من هذا المستند مش تغطية أكبر عدد من الميزات، بل إثبات القدرة على بناء نظام صغير نسبيًا لكن سليم معماريًا بالكامل — وهو المعيار الذي يميز المشاريع القوية في معارض التخرج ومقابلات العمل التقنية.